# Technical Report: AI-Driven Network Intrusion Detection System

**Prepared for:** Megaminds IT Services

**Position:** Cybersecurity and Network Security Research Intern

**Candidate:** [Vinodhkumar]

## 1. Problem Overview

Modern network environments are constantly evolving, and so are the tactics of malicious actors. While traditional signature-based Intrusion Detection Systems (IDS) remain a cornerstone of network defense—offering high precision against known exploits—they possess a critical blind spot: they cannot detect zero-day vulnerabilities or polymorphic threats that deviate from established signatures. As attackers increasingly utilize evasive techniques, reliance on purely signature-based mechanisms leaves organizational networks vulnerable.

To address this gap, modern cybersecurity architecture requires behavioral analysis. The objective of this project is to conceptualize, design, and implement an AI-driven network traffic analysis prototype. This system must be capable of ingesting network telemetry, extracting critical mathematical features, and leveraging Machine Learning (ML) to classify traffic flows as benign or malicious, effectively acting as an anomaly-based detection engine capable of supplementing traditional signature defenses.

## 2. Objectives

The primary objectives of this assessment are to:

1. **Architecture Design:** Develop a robust, flow-based IDS prototype capable of continuous, batch-based traffic ingestion.
2. **Feature Engineering:** Extract a highly optimized subset of mathematical features from raw network traffic to accurately differentiate standard operations from attack patterns.
3. **Threat Classification:** Train and deploy an ML model (Random Forest) to classify 15 distinct traffic profiles, including baseline normal traffic and 14 specific attack vectors (e.g., PortScans, DDoS, Brute Force).
4. **Operational Explainability:** Implement a true Machine Learning explainability engine that extracts real-time feature importance weights to justify flagged anomalies to a Security Operations Center (SOC) analyst.
5. **Simulated Deployment:** Prove the efficacy of the system through a command-line interface (CLI) that processes sequential traffic and generates contextual alerts.

## 3. Traffic Data / Dataset Description

To train the anomaly detection engine, this project utilized the widely recognized **CIC-IDS-2017** dataset from the Canadian Institute for Cybersecurity.

This dataset was selected because it represents a highly realistic, modern network environment. It includes background benign traffic (HTTP, HTTPS, FTP, SSH) alongside modern attack scenarios generated over a five-day capture period.

The raw dataset comprises several gigabytes of PCAP-derived CSV files. For this implementation, the data was rigorously cleaned and concatenated into a unified dataset comprising over **2.8 million validated network flows**.

The system was trained to identify the following mandatory and supplementary traffic profiles:

- **BENIGN:** The standard operational baseline.
- **PortScan:** Reconnaissance activity indicative of an attacker mapping the network.
- **DDoS (Distributed Denial of Service):** Volumetric attacks designed to overwhelm server resources.
- **FTP-Patator / SSH-Patator:** Aggressive brute-force credential attacks.
- **Infiltration:** Advanced, low-volume activity representing internal lateral movement or malware drop.
- *Additional Classes:* DoS variants (Hulk, GoldenEye, Slowloris), Web Attacks (XSS, Brute Force, SQL Injection), Botnet traffic, and Heartbleed exploits.

# 4. System Architecture

The prototype is engineered as a robust Python-based pipeline, structured to simulate a production environment.

1. **Data Ingestion & Sanitization Layer (merge_datasets.py):**
    - This component processes the raw CIC-IDS-2017 CSV files. It resolves known dataset formatting issues (e.g., invisible whitespace in column headers) and sanitizes the data by removing NaN (Not a Number) and Infinity values caused by packet division errors, which would otherwise crash the ML engine.
2. **Model Training Engine (classifier.py):**
    - This script handles the extraction of the target features, encodes the text-based labels, splits the data (70/30), and trains the core Random Forest algorithm. It exports the trained model and label encoders as binary .pkl artifacts.
3. **Real-Time Inference & Simulation CLI (run_scenarios.py):**
    - This acts as the operational deployment. It streams network traffic in chunks, maintains a rolling deque buffer of benign history, and evaluates sequential batches of flows. If an anomaly threshold is breached, it triggers a critical alert and extracts the model's decision-making weights.

# 5. Feature Extraction Methodology

Analyzing raw payload data (Deep Packet Inspection) is computationally expensive and raises

privacy concerns. Therefore, this system relies entirely on flow-based statistical analysis.

Rather than training on the 70+ features present in the raw CIC-IDS-2017 dataset—which causes overfitting and high computational overhead—the model was rigorously optimized to utilize a synchronized **14-Feature Vector**. These features capture the mathematical "shape" of a network conversation:

1. **Destination Port:** Essential for identifying targeted services (e.g., Port 21 for FTP brute force, Port 80 for web attacks).
2. **Flow Duration:** Differentiates rapid, high-volume attacks (DDoS) from low-and-slow tactics (Infiltration).
3. **Total Fwd/Bwd Packets & Total Lengths:** Measures the sheer volume and directionality of data transfer.
4. **Flow Bytes/s & Packets/s:** The velocity of the traffic. Unusually high packet velocities are a primary indicator of volumetric flooding.
5. **Packet Length Mean & Average Packet Size:** Identifies abnormal payload delivery or data exfiltration attempts.
6. **Flag Counts (SYN, ACK, PSH, RST):** Tracks TCP state anomalies. For example, high SYN counts correlate strongly with network scanning and reconnaissance.

# 6. Detection Methodology

To classify the extracted features, this architecture implements a **Random Forest Classifier**.

- **Algorithmic Justification:** While Deep Learning (e.g., Neural Networks) is popular, it often functions as a "black box," making it difficult for an analyst to understand *why* an alert was generated. Random Forest, an ensemble learning method utilizing multiple decision trees, is highly resistant to overfitting on imbalanced network data and, crucially, provides native **Feature Importance** scoring.
- **Hyperparameter Tuning:** The model was configured with n_estimators=100 (for classification stability), max_depth=20 (to prevent the model from memorizing noise while capturing complex attack patterns), and class_weight="balanced_subsample" (to ensure the model did not ignore low-volume attacks like Infiltration in favor of high-volume attacks like DDoS).

# 7. Testing, Evaluation, and Results

The system was evaluated using a rigorous 30% holdout testing set (848,423 flows). The model demonstrated exceptional performance across the multi-class environment.

## Aggregate Performance Metrics

- **Overall Accuracy:** 98.41%
- **Precision (Weighted Average):** 99.58%
- **Recall (Weighted Average):** 98.41%
- **F1-Score (Weighted Average):** 98.95%

### Scenario Demonstration Results

The run_scenarios.py CLI successfully demonstrated the model's capability in a simulated operational environment:

- **Scenario 1: Normal Traffic Baseline:** The engine processed standard background traffic seamlessly, buffering the flows and reporting no sustained campaign alerts, effectively demonstrating a zero false-positive rate during the baseline test.
- **Scenario 2: Reconnaissance (PortScan):** The model immediately flagged the isolated reconnaissance packets. However, the alerting engine correctly identified that the volume remained below the critical threshold, demonstrating the system's ability to reduce alert fatigue by distinguishing between isolated anomalies and sustained campaigns.
- **Scenario 3: Denial of Service (DDoS):** The system ingested a batch of attack flows and triggered a CRITICAL ALERT. The Explainability Engine pulled the true ML weights, indicating that the model flagged the attack primarily based on the Destination Port (80) and massive Flow Packets/s anomalies.
- **Scenario 4: Additional Attack (FTP-Patator):** The model successfully identified a sustained brute-force attack against Port 21, correctly flagging 18 out of 25 flows in the tested batch and triggering a campaign alert.
- **Scenario 5: Ambiguous Case (Infiltration):** The model detected the infiltration attempt, but the confidence and subsequent volume fell below the campaign alerting threshold. This highlights a known limitation (discussed below) regarding the difficulty of detecting low-volume, highly evasive lateral movement using purely statistical metrics.

# 8. Limitations

1. **Encrypted Payloads & Mimicry:** Because the system evaluates flow statistics rather than packet contents, an attacker utilizing an established, encrypted tunnel (e.g., SSH tunneling) to manually exfiltrate data at normal human speeds could evade the mathematical thresholds.
2. **Ambiguous Class Overlap:** The model occasionally misclassifies structurally similar attacks. For example, aggressive web scraping might theoretically be flagged as a Web Attack or DoS if the packet velocity and size profiles overlap significantly with known malicious signatures.
3. **Concept Drift:** ML models degrade over time as attackers evolve their tools. The model is currently optimized for 2017-era attack profiles; continuous retraining on newer datasets (e.g., CIC-IDS-2019 or live internal data) is required for production deployment.

# 9. Future Improvements

To transition this prototype into a production-ready enterprise solution, I propose the following enhancements:

- **Hybrid Integration:** Deploy this Machine Learning engine in parallel with a signature-based IDS (like Snort or Suricata). The signature engine would instantly block

known threats with near-zero computational overhead, allowing the ML model to focus exclusively on behavioral analysis for zero-day threats.

- **Time-Series Deep Learning:** Transition from Random Forest to Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) models. These networks analyze the sequential *order* of packets over time, significantly improving the detection of slow, methodical Infiltration or Advanced Persistent Threats (APTs).
- **SIEM Output:** Modify the CLI to export JSON-formatted logs directly into a Security Information and Event Management (SIEM) tool (e.g., Splunk, ELK Stack) to enable centralized monitoring and automated incident response workflows.

# 10. Project Deliverables

- **GitHub Repository:** https://github.com/vinodhkumar232/megaminds-ids-assessment
- **Video Demonstration:** https://drive.google.com/file/d/189ytbg95KeodqyyvUSriqp9rFOeyXtJK/view?usp=sharing

*End of Report*